



Sandya Mannarswamy

CODE SPORT

In this month's column, we will look at the concept of dynamic software update solutions, which allow even an operating system to be updated without requiring a reboot.

In last month's column, we had looked at software upgrade failures, discussing the kind of failures that can arise during the upgrade process, and how such failures can be reduced. Complex software upgrades, such as patching an operating system kernel, typically require planned downtime in which the required patches are applied and the system is rebooted. However, even this planned downtime leads to loss of service. There has therefore been considerable research of late on Dynamic Software Update (DSU) solutions, which do not require a system reboot or cause loss of application availability. In this month's column, we examine the issues involved in developing software updates/patches that can be dynamically applied; how to ensure the safety of DSUs; and what are the existing DSU solutions.

How does DSU work?

Consider any software update. It consists of changes to parts of an existing software application. In the context of the C language, a software program consists of functions and data. A software update can contain changes (addition/deletion/modification) to functions, to data, or to both. Let us consider a simple example of a security patch that involves a change to a function 'foo', whose original version is shown in the snippet below:

```
int foo(char* app_buffer, char* input_string)
{
    ...
    strcpy(app_buffer, input_string);
}
```

Since the unsafe 'strcpy' call can lead to security vulnerabilities like buffer overflows, the developer decided to fix this issue by replacing 'strcpy' with 'strncpy', which copies only as many characters as allocated for 'app_buffer', avoiding buffer-overflow errors.

To construct a DSU for a statically patched application executable, we first need to know what the changes are between the old and new versions of the software. In this case, we know that it is the function 'foo' that has been changed. Once the changes are determined, the DSU solution generates the executable (object) code for the changed part (foo).

Now, on production machines that run the application, before the DSU can be applied, we need to determine whether the application is in a state in which it is safe for the update to be done. If any active application threads are executing the old version of 'active functions' (functions that have changed), then it is safer to wait until they finish executing these before patching. In our case, let's wait for any thread executing 'foo' to return from it, before applying the update. Later, we will discuss whether merely waiting for 'active functions' to exit constitutes sufficient safety for DSUs.

Once you determine that it's safe, apply the DSU. One way is to overwrite the entry point of each modified function with an instruction that jumps to the new version of the function. This is what we do. Now, the next thread that tries to execute 'foo' will encounter the jump instruction, and go to the new version.

We have to ensure that the DSU happens atomically—either all 'active functions' are